

Proje Raporu & AI Engineering Blueprint

Takım-tabanlı sosyal MVP — Next.js 15 (App Router) + Supabase · Senior Full-Stack / AI-Native Tech Lead

Teslimat: #2 README tamamlayıcısı + #3 AI Engineering Blueprint **Stack:** Next.js 15 · Supabase (Postgres/Auth/Realtime) · RLS · Vercel-hazır

Durum: İnşa edildi & doğrulandı (tsc + build + lint yeşil)

Yönetici özeti

Kiracının (tenant) **kişi değil takım** olduğu bir **takım-tabanlı sosyal medya MVP**'si: bir kullanıcı tam olarak bir takıma aittir, bireysel profil yoktur ve her eylem — gönderi, takip, mesajlaşma — takım kimliği altında gerçekleşir. **Tüm yetkilendirme veritabanında Row-Level Security (RLS) ile zorlanır**; uygulama katmanı asla güvenlik sınırı değildir.

9 / 9

KARŞILANAN FONKSİYONEL ALAN

85

COMMIT'LENEN DOSYA

7 + 3

SQL MİGRATION + PGTAP TESTİ

3

BAĞIMSIZ AI ARACI

İnşa süreci **önce-plan, araştırmayla-desteklenmiş, çoklu-ajan** bir akışla ilerledi ve her AI çıktısı güvenilir bir taslak gibi ele alındı. **Üç inceleme turu** (biri AI'nın planına, biri 3-eleştirmenli adversarial tur, biri de inşa edilen koda) **gerçek, teslimi engelleyen hatalar** yakaladı — \$6'da listelendi. Sonuç: **tsc** + **next build** (13 route) + **next lint** tamamen yeşil.

Dürüst durum. Zorunlu fonksiyonel kapsam, README ve AI Engineering Blueprint **tamamlandı ve doğrulandı**. Kalanlar: (1) private GitHub deposuna push (adayın hesabı + mehmetduvarci2011@gmail.com davetini gerektirir); (2) *opsiyonel* Vercel deploy ve video. Doğrulama; type-check / build / lint / yazılmış-pgTAP / adversarial-review seviyesinde yapıldı — canlı uçtan uca çalıştırma, sağlanmış bir Supabase projesi gerektirir (env + Auth Hook), README'de belgelenmiştir.

1. Kapsam uyumu — case dökümanına göre

Her gereksinim alanı, kodda nerede yaşadığıyla eşleştirildi. "Karşılandı" = uygulandı, uçtan uca bağlandı ve yeşil build'de derleniyor.

ALAN	GEREKSİNİM (CASE'DEN)	DURUM	NEREDE
1. Auth & oturum	Email+şifre, Google OAuth (PKCE), kalıcı oturum, net giriş/çıkış durumu, giriş sonrası kullanıcı→takım bağı	Karşılandı	app/(auth) , auth/callback , middleware.ts , site-header
2. Takım modeli	Bir kullanıcı → bir takım, paylaşılan yetkiler, public/private, roller kapsam dışı, signup'ta takım seed'i, içerik takıma kapsamlı	Karşılandı	handle_new_user trigger, profiles.id PK=FK, RLS
3. Gönderi	Her üye takım adına gönderebilir; gönderiler takıma ait; metin + oluşturulma zamanı; async/hata yönetimi	Karşılandı	actions/posts.ts , composer , post-card
4. Takip	Kendini takip yok; tek yönlü; public=anında; private=istek→onay/red; takip/takipten-çık; istekler saklanır	Karşılandı	actions/follows.ts , app/requests , follows tablosu
5. Mesajlaşma	Takım↔takım, her üye takım adına, kendine mesaj yok, başlat/gönder/al, en-yeni-mesaja-göre sıralı liste, kalıcı	Karşılandı	actions/messages.ts , get_inbox , Realtime message-thread
6. Ana feed	Public gönderiler herkese (girişsiz dahil); private yalnızca onaylı takipçiye; en-yeni-önce; temiz sorgulama	Karşılandı	app/page.tsx , get_feed RPC, anon RLS
7. Veri modeli & güvenlik	Yalnızca kendi takımı adına eylem; tüm varlıklar kapsamlı; private görünürlük zorlanmış; şema+RLS açıklanmış; aşırı soyutlama yok	Karşılandı	migrations/0005_rls.sql , docs/plan/02 , pgTAP
8. Mimari (README)	Yüksek-seviye mimari, ERD, temel trade-off/alternatifler, riskler/önlemler	Karşılandı	README.md \$8/\$6.1/\$14/\$15 (Mermaid)
9. Yapı & kalite	Next.js App Router, Supabase, temiz async/hata, makul klasör yapısı, okunur kod, modern-tutarlı UI	Karşılandı	tüm repo; ActionState konvansiyonu; koyu UI kit

Teslimatlar

TESLİMAT	DURUM	NOTLAR
1. GitHub deposu	Push bekliyor	Yerelde commit'lendi (ilk commit, 85 dosya). Private depoya push + <code>mehmetduvarci2011@gmail.com</code> daveti; ≥30 gün private tut.
2. README.md	Karşılandı	Kurulum, şema, RLS özeti, mimari, ERD, varsayımlar/trade-off, bilinen limitler, iyileştirmeler.
3. AI Engineering Blueprint	Karşılandı	Bu döküman + <code>docs/AI_ENGINEERING_BLUEPRINT.md</code> + <code>CLAUDE.md</code> + <code>docs/plan/</code> + memory dosyaları.
4. Canlı deploy	Opsiyonel	Deploy edilmedi. Vercel runbook README'de; uygulama Vercel-hazır.
5. Video anlatım	Opsiyonel	Kaydedilmedi.

2. Kullanılan AI araçları — kim ne yapar, neden

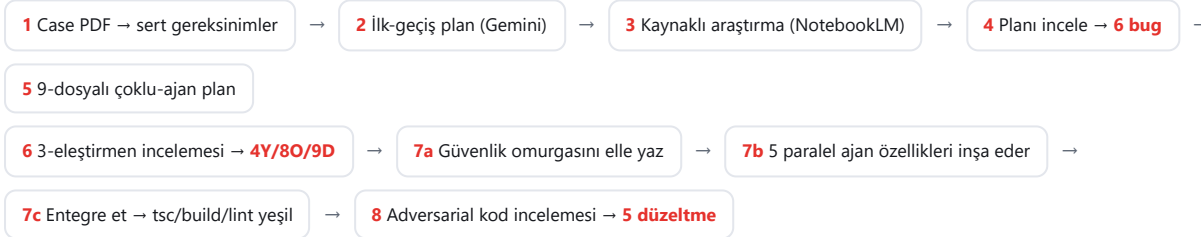
Bilinçli bir çoklu-model ayrımı: her model, hata modunun en ucuz olduğu yerde kullanılır.

ARAÇ	ROL	NEDEN BU ARAÇ
Claude Code (Opus 4.8)	Ana agentic sürücü. Migration'ları, RLS'i, Server Action'ları, bileşenleri yazdı; <code>tsc</code> / <code>build</code> / <code>lint</code> çalıştırdı; çoklu-ajan fan-out'u ve adversarial review'u yönetti.	Dosyaları düzenleyen ve toolchain'i tek döngüde çalıştıran terminal-native ajan — üretilen SQL/TS körlemesine yapıştırılmaz, derlenir ve incelenir. Her turda commit'li bir ruleset okur.
Gemini (3.5 Flash)	İlk-geçiş mimari taslağı: ERD, DDL taslağı, RLS taslağı, middleware, action iskeletleri.	Sistemin <i>şekli</i> için hızlı, uzun-bağlamlı beyin fırtınası — mühendislik zamanı harcamadan önce ucuz iterasyon. Denetlenecek bir öneri olarak ele alındı.
NotebookLM	Kaynağa dayalı saha araştırması: case PDF'i + seçili Supabase/Next.js dokümanları kaynak olarak, bir LLM'in ince şekilde yanlış yaptığı kararlara yönelik hedefli sorular.	Cevaplar yüklenen kaynaklara bağlı (citation-bound) , riskli kararlarda halüsinasyon riskini keskin biçimde düşürür (RLS özyineleme, JWT tazeliği, anon NULL-karşılaştırması, keyset vs OFFSET).

Neden çoklu-model, tek model değil: Gemini hızlı yayılır (harika genişlik, zayıf doğruluk) → tek-kullanımlık ilk taslak. NotebookLM tutucu ve kaynaklı (harika doğruluk, dar) → riskli kararları notlandırır. Claude Code, kodu derleyip incelemeleri geçirerek döngüyü kapatır. Devir teslimler bilinçlidir.

3. Geliştirme akışı

Proje bir **önce-plan, araştırmayla-desteklenmiş, çoklu-ajan fan-out** olarak ilerledi — "prompt → yapıştır → umut et" değil.



Neden önce-plan-sonra-uygula

3 günlük bütçe, işi tekrar yapmamayı ödüllendirir. Sabitlenmiş bir plan + paylaşılan isimlendirme/arayüz kontratı, bağımsız üretilen bölümlerin sürüklenmek yerine yakınsamasını sağlar — 5 paralel özellik ajanı, **ilk entegrasyon geçişinde birlikte derlenen ayrık dosyalar** yazdı; sıfır arayüz yeniden-adlandırması.

Değerin olduğu yer

İki aşama: **4 & 6. aşamalar** (plana asla güvenilmez — kaynaklı araştırmaya ve bağımsız eleştirmenlere karşı diff'lenir) ve **8. aşama** (*kod* da aynı güvenilmez-taslak muamelesini görür). Çoğu hatayı yazar değil, toolchain ve eleştirmenler yakaladı.

4. Agentic ruleset & talimat dosyaları

Gerçek, commit'li artefaktlar (case'in "faydalı dosya örnekleri") — düz metin değil.

ARTEFAKT	AMAÇ
<code>CLAUDE.md</code>	Her turda okunan işletim ruleset'i: ürün değişmezleri, kilitli kararlar, sert kodlama kuralları (<code>await createClient()</code>), konuşmalarda insert-yerine-upsert, sadece-status follow güncellemesi, <code>ActionState</code> şekli, <code>revalidateTag</code>), yetkili paylaşılan isimlendirme ve bir "bitti tanımı" + adversarial öz-inceleme kontrol listesi. Önceki bug'ları kodlar, böylece ajan onları geri getiremez.
<code>docs/plan/CONTRACTS.md</code>	Paralel uygulama için entegrasyon kontratı — kesin dosya sahipliği, export imzaları, import yolları, konvansiyonlar, RPC isimleri. 5 ajanın sürüklenmeden paralel inşa etmesini sağlayan şey budur.
<code>docs/plan/00-09</code>	Kalıcı tasarım, alan başına bir sahip; <code>00-overview.md</code> yetkilidir ve plan incelemelerinin bulunduğu her hatanın kanonik çözümünü (satır içi düzeltilmiş SQL/TS ile) kaydeder.
Proje memory dosyaları	Bağlam sıfırlamalarını atlatan ve ajanı hizada tutan kalıcı, oturumlar-arası notlar (kapsam, kilitli kararlar, önceki-plan bug'ları).

5. Prompt & bağlam stratejisi

- **Kısıt-öncelikli prompt'lar.** Her görev, değişmezle başlar ("Tenant = Takım, profil yok, RLS güvenlik sınırır") — böylece model gerçek kutunun içinde optimize eder. Genel "bir sosyal feed yap" prompt'larından kaçınıldı; onlar çekirdek modeli ihlal eden kullanıcı-merkezli şemalar üretir.
- **Riskli kararlar için kaynaklı > üretken.** Doğruluk uçurumu olan her şey, önce NotebookLM tarafından kaynaklara karşı cevaplandı, sonra kodlama ajanına yerleşmiş bir karar olarak verildi.
- **Bağlama enjekte edilen paylaşılan isimlendirme/arayüz kontratı.** Plan dosyaları + `CONTRACTS.md` genelinde aynı tanımlayıcılar → bağımsız üretilen bölümler birlikte derlenir.
- **Döngüde-derle-ve-incele.** Ajan, düzenlemelerden sonra `tsc` / `build` / `lint` çalıştırır ve hataları geri besler; ardından bağımsız eleştirmen ajanlar sonucu inceler.
- **Adversarial öz-inceleme.** Sabit bir talimat, diff'i bir saldırgan gibi yeniden okumaya zorlar (" `anon` ne görür? bir çağrı başka takımın id'sini geçebilir mi?") — bu, her-bulguya-çürütme adımlı çoklu-ajan inceleme akışı olarak kurumsallaştırıldı.

6. AI çıktısı nasıl incelendi & doğrulandı — somut kanıt

Her biri **gerçek** hatalar yakalayan üç bağımsız inceleme turu. Bu, AI çıktısının denetlendiğinin — güvenilmediğinin — kanıtıdır.

Tur 1 — Gemini ilk-geçiş planının incelemesi (6 bug, kod-öncesi)

BUG	NEDEN BOZAR	DÜZELTME
<code>onboarded</code> <code>teams</code> 'e yazılıyor ama middleware <code>app_metadata</code> 'yı okuyor	Hep <code>undefined</code> → sonsuz <code>/onboarding</code> yönlendirmesi	<code>onboarded</code> 'ı Auth Hook ile JWT'ye enjekte et; middleware aynı claim'i okur
<code>getOrCreateConversation</code> çıplak <code>.insert()</code> kullanıyor	Konuşma zaten varsa <code>23505</code> fırlatır	<code>upsert(..., {onConflict, ignoreDuplicates})</code> sonra select
<code>raw_app_meta_data</code> yazan trigger ile claim enjeksiyonu	Token mint'iyile yarışır; ilk token <code>team_id</code> 'siz olabilir	Custom Access Token Auth Hook, mint anında <code>team_id</code> okur
Inbox en son mesaja göre sıralanmıyor	"En yeni mesaj zamanına göre sıralı" başarısız	<code>LEFT JOIN LATERAL max(created_at)</code>
<code>createClient()</code> await edilmiyor (Next 15)	<code>cookies()</code> async → auth sessizce null	Her yerde <code>await createClient()</code>
<code>follows</code> UPDATE'i kolon-kısıtlı değil	Onaylayan FK kolonlarını yeniden yazabilir	<code>REVOKE UPDATE; GRANT UPDATE(status)</code>

Tur 2 — Claude'un yazdığı planın 3-eleştirmen incelemesi (4 yüksek / 8 orta / 9 düşük)

Aynı güvenilmez-taslak disiplini, kapsamlı planın kendisine uygulandı. Yüksekler arasında:

- JWT claim-yolu uyuşmazlığı** (okuyucular `app_metadata.team_id` ile başka yollar arasında bölünmüş — RLS'in %100'ünü bozardı).
- Auth Hook'un RLS altında 0 satır okuması** (`supabase_auth_admin` rolünün `profiles / teams` üzerinde okuma politikası yok → kimse giriş yapamaz).
- Eksik `posts.team_id` default'u** (böylece `insert({content})` `NOT NULL` 'dan geçemezdi).
- `get_feed` kontrat sapması** ve **private-takım isim görünürlüğü** açığı.

Hepsi `00-overview.md` `§6` 'da düzeltilmiş SQL/TS ile kanonik olarak çözüldü.

Tur 3 — inşa edilen kodun adversarial incelemesi (4 eleştirmen boyutu + her-bulguya çürütme adımı → 5 doğrulandı, 0 yanlış-pozitif)

ÖNEM	KUSUR	DÜZELTME
Güvenlik	<code>check_team_follows()</code> (<code>SECURITY DEFINER</code>) varsayılan <code>PUBLIC</code> execute grant'ını çağırıcı-koruması olmadan tuttu → herkes, anon dahil, private takip grafiğini PostgREST RPC ile yoklayabilirdi.	Çağırıcı-koruması (yoklama yalnızca çağırıcının kendi takımıyla sınırlı) + <code>PUBLIC</code> /anon grant iptali. <i>Dikkat: doğrulayıcının ilk-önerdiği düzeltme, fonksiyonu <code>authenticated</code> olarak çağıran <code>posts</code> RLS politikasını bozardı; uygulanan düzeltme bunu hesaba katar.</i>
Veri/UX	Red sonrası, hayatta kalan <code>rejected</code> satırı yeniden-isteği çakıştırdı (<code>23505</code>), sahte "başarı" olarak yutuldu → kalıcı kilitleme.	Reddedilmiş tombstone'un RLS-izinli sil-sonra-ekle'si (bir <code>upsert</code> olmayan follower-tarafı UPDATE politikasına takılırdı).
Düşük	Feed dilimleri ayrı ayrı <code>PAGE_SIZE</code> ile sınırlanıp birleştiriliyor → 2× kadar, global en-yeni-N değil.	Birleştirilmiş sonucu <code>FEED_PAGE_SIZE</code> 'a dilimle.
Düşük	"Yeni gönderiler" hapi, tıklamasız yenilemeden sonra bayat sayı gösterebilir.	Baz çizgisi değiştiğinde sayacı sıfırla.
Düşük	<code>completeOnboarding</code> bir <code>refreshSession()</code> hatasını yok saydı → <code>/onboarding</code> 'e geri sekme olasılığı.	Hatayı yakala ve bayat token ile yönlendirmek yerine tekrar-dene sun.

İnsanın transkripsiyon sırasında yakaladığı bir bug. Migration'ları elle yazarken planda bir **bağımlılık-sıralama hatası** ortaya çıktı: `posts.team_id`, `current_user_team_id()` 'ye default veriyor ama plan bu fonksiyonu tablodan iki migration sonra oluşturuyordu — yani temiz bir `supabase db reset` 'te `CREATE TABLE posts` başarısız olurdu. Tablosuz JWT çözücüyü `0001` 'de, tablolardan önce tanımlayarak düzeltildi.

Neyin gerçekten yakaladığına göre sıralanmış doğrulama yöntemleri: (1) kaynaklı-araştırma diff'i; (2) çürütme adımlı bağımsız adversarial eleştirmenler (RPC sızıntısı + yeniden-istek kilidi); (3) `tsc` / `build` (incelemeden önce yeşil); (4) gereksinim geri-izleme. **Akışa kodlanan ders: AI genişliği mükemmel; AI doğruluğu mekanik olarak doğrulanmalı.**

7. Temel kararlar — aday vs. AI

KARAR / ARTEFAKT	SÜRÜCÜ
Çekirdek alan modeli (Tenant=Takım, profil yok, public/private, istek-olarak-takip)	Aday — case'den okundu; pazarlıksız çerçeve.
"MakerKit kullanma; minimal inşa et"	Aday — hesap/faturalama/rol modeli, bir-kullanıcı-bir-takım + aşırı-soyutlama-yok ile çelişir.
İlk-geçiş ERD / DDL / RLS taslağı	AI (Gemini) — hız taslağı, denetlendi.
Riskli-karar araştırması (RLS özyineleme, JWT claim'leri, anon RLS, cache, inbox sıralama)	AI (NotebookLM), aday-çerçevesi — soruları aday yazdı.
Bug'ları yakalamak (üç turun tümü)	Aday-önderliğinde, AI-destekli — belirleyici disiplin.
Kilitli düzeltmeler (Auth Hook, upsert, await client, kolon-GRANT, lateral-join inbox, denormalize is_public)	Aday — nihai mimari kararlar.
Uygulama (migration'lar, RLS, action'lar, bileşenler, testler)	AI (Claude Code), aday-inceleme — CLAUDE.md kısıtları altında, build + adversarial review ile kaplandı.

Tek cümlelik özet: aday modele, kısıtlara ve düzeltmelere sahiptir; AI genişliğe, taslağa ve mekanik yürütmeye sahiptir; ve her AI artefaktı sayılmadan önce kaynaklı araştırma + çalışan bir toolchain + bağımsız adversarial review ile kaplanır.

8. Mimari & veri modeli (özet)

Şekil

Vercel üzerinde tek bir Next.js 15 uygulaması ↔ tek bir Supabase projesi. Tarayıcı asla ayrıcalıklı kimlik bilgisi tutmaz: okumalar RLS-korumalı RSC, yazmalar Server Action'lardan geçer ve Realtime tek canlı kenardır (mesajlaşma). Özel API katmanı yok.

Kiracı çözümü

Aktif **team_id**, JWT **app_metadata** içinde yaşar; **Custom Access Token Auth Hook** ile enjekte edilir — *ilk* token'da mevcut olduğundan RLS, kiracı kapsamını sıfır DB gidiş-dönüşü ve refresh dansı olmadan çözer.

Altı tablo

teams (tenant) · **profiles** (auth.users → tek takım; PK=FK bir-kullanıcı-bir-takımı zorlar) · **posts** (takıma ait; **is_public** takımdan denormalize, trigger-korumalı) · **follows** (tek tablo + **status** enum; bileşik PK; **CHECK(follower*following)**) · **conversations** (simetrik, **team_a<team_b** + **UNIQUE** → çift başına tek satır, kendine-sohbet imkânsız) · **messages** (gönderen bir takım).

Önemli güvenlik mekanizmaları

- anon vs authenticated ayrık politikalar** private yüklemi anonim sorgu planından tamamen dışarıda tutar — çıkış yapmış bir ziyaretçiye asla private içerik sunulamaz.
- SECURITY DEFINER RPC'ler** (**get_feed** / **get_inbox** / **get_incoming_follow_requests**) **caller == current_user_team_id()** doğrular, **search_path=''** sabitler ve private karşı-taraf isimlerini güvenle yüzeye çıkarır — tablo RLS'i savunma-derinliği olarak açık kalır.
- follows üzerinde kolon-seviyesi GRANT UPDATE(status)** onay/red'in FK kolonlarını yeniden yazmasını imkânsız kılar — RLS satırları, bu kolonları kaplar.
- pgTAP RLS testleri** anon-vs-private feed izolasyonunu, follow kolon-kilidini ve konuşma izolasyonunu doğrular.

9. Trade-off'lar, limitler & sonraki adımlar

Temel trade-off'lar

- MakerKit yerine kendi minimal kurulumumuz (rolleri/faturalaması modelle çelişir).
- Feed canlılığı realtime yerine revalidation + hap ile (daha küçük etki alanı).
- Hibrit feed: **posts** üzerinde RLS emniyeti + hızlı **get_feed** RPC.
- Lateral-join inbox sıralaması (MVP ölçeğinde sıfır yazma-amplifikasyonu).

Bilinen limitler / sonraki

- Feed canlılığı public gönderiler için push değil poll (~15sn).
- Takım değiştirme / roller / okundu bilgisi / medya yok (kapsam gereği).
- Realtime feed (Broadcast-from-DB), denormalize **last_message_at**, keyset sonsuz kaydırma — belgelenmiş ölçek yolları.
- CI'a bağlı daha geniş Playwright E2E.

Riskler & önlemler (seçilmiş)

RİSK	ÖNLEM
Private gönderilerin paylaşılan cache'e sızması	Tek dinamik feed route; private dilim yalnızca <code>cookies()</code> sonrası çekilir; public dilim private-olmayan bir tag altında cache'lenir.
<code>get_feed</code> ayrıcalık yükseltmesi	Fonksiyon-içi <code>_viewer_team_id == current_user_team_id()</code> + <code>search_path=''</code> ; <code>posts</code> üzerinde RLS emniyeti.
Anon'un private veriye ulaşması	Özel <code>TO anon USING(is_public=true)</code> ; hassas tablolarda açık <code>REVOKE</code> ; <code>check_team_follows</code> grant'ı kilitlendi.
Sonsuz onboarding yönlendirmesi	<code>onboarded</code> JWT'ye enjekte edilip aynı claim'den okunur; onboarding sonrası <code>refreshSession()</code> .

10. Nasıl çalıştırılır & teslim kontrol listesi

Çalıştır (yerel)

```
pnpm install → cp .env.example .env.local (Supabase anahtarlarını doldur) → supabase start → supabase db reset (migration 0001–0007 + seed) → pnpm dev . Ardından Auth Hook'u etkinleştir ve Confirm email'i kapat (README §4). Kapılar: pnpm typecheck · pnpm build · pnpm lint · supabase db test .
```

Teslim için

- Private** bir GitHub deposu oluştur; mevcut commit'i push et.
- `mehmetduvarci2011@gmail.com` 'u davet et; ≥30 gün private tut.
- (Opsiyonel) Vercel deploy → canlı linki README'ye ekle.
- (Opsiyonel) 5–10 dakikalık anlatım videosu kaydet.